

Informe de Implementación: CRUD de Clientes

Proyecto Web Crustáceo Caribeño • Desarrollo Web • Semana 8 • Pontificia Universidad Javeriana

DOCUMENTO DE EQUIPO

1. Contexto y Objetivo

En la Semana 8 y 9 del curso de Desarrollo Web se solicitó la construcción del frontend en **Angular** conectando los conceptos aprendidos con los datos del backend de Spring Boot. Como aún no se ha integrado la comunicación vía API REST (tema de la Semana 10), el requerimiento especificaba:

- Crear las clases/interfaces TypeScript para todas las entidades que estaban en Spring Boot.
- Pasar el CRUD de Comidas a Angular con datos quemados.
- **Implementar el CRUD de Clientes con datos quemados**, manteniendo la misma coherencia arquitectónica y funcional.

Este informe documenta la implementación completa del módulo de Clientes en el proyecto `restaurante`, realizada siguiendo estrictamente el patrón oficial explicado por el profesor Juan Sebastián Angarita en su video de clase y repositorio de referencia (`8.Angular`).

2. Alineación con el Patrón Oficial del Profesor (8.Angular)

Para asegurar que el código no presente inconsistencias frente a la rúbrica y explicaciones del profesor, se tomó como base su ejemplo de `Student` y se trasladó de forma directa a la entidad `Cliente`:

Aspecto / Componente	Repositorio del Profesor (8.Angular)	Implementación en Crustáceo Caribeño
Modelo / Interfaz	<code>Student</code> (id, name, age, semester, correo...)	<code>Cliente</code> (idCliente, nombre, apellido, correo, telefono, direccion, password, activo)
Servicio con datos quemados	<code>StudentService</code> con arreglo privado <code>studentArray</code>	<code>ClienteService</code> con arreglo privado y 10 clientes pre-cargados
Coherencia de datos iniciales	4 estudiantes ficticios de muestra	Los 10 clientes exactos definidos en <code>DataLoader.java</code> de Spring Boot
Formulario Reactivo	<code>FormGroup</code> con validadores y detección <code>isEdit</code> en <code>ngOnInit</code>	<code>ClienteFormComponent</code> con <code>FormBuilder</code> , validadores y <code>patchValue</code> si hay ID
Manejo de estado / Borrado	Eliminación lógica con cambio de estado / arreglo	Eliminación lógica mediante <code>activo: false</code> y reactivación disponible en la tabla
Vistas del Módulo	Tabla, Formulario (Crear/Editar) y Detalle	<code>clientes-tabla-page</code> , <code>cliente-form</code> y <code>cliente-detail</code>

Coherencia con Spring Boot: Los datos iniciales no son aleatorios; se replicaron los 10 clientes creados en la semilla del backend (Juan Rodríguez, María Gómez, Carlos López, Ana Martínez, Luis García, Diana Sánchez, Andrés Pérez, Laura Ramírez, Jorge Herrera y Sofía Castro). Esto facilitará enormemente la migración a REST la próxima semana.

3. Resumen de Archivos Añadidos y Modificados

Tipo	Ruta del Archivo	Propósito y Contenido
AÑADIDO	<code>src/app/service/cliente.service.ts</code>	Servicio singleton inyectable con los 10 clientes quemados y los métodos CRUD completos.
AÑADIDO	<code>src/app/pages/clientes-tabla-page/*</code> (.ts, .html, .scss)	Página con la tabla de administración de clientes, estados visuales (Activo/Inactivo) y botones de acción.
AÑADIDO	<code>src/app/pages/cliente-form/*</code> (.ts, .html, .scss)	Formulario reactivo dual (alta y edición) con validaciones en tiempo real para todos los campos.
AÑADIDO	<code>src/app/pages/cliente-detail/*</code> (.ts, .html, .scss)	Ficha detallada del cliente con información de contacto, contraseña enmascarada y accesos rápidos.
MODIFICADO	<code>src/app/app.routes.ts</code>	Registro de las rutas <code>/clientes</code> , <code>/clientes/add</code> , <code>/clientes/update/:id</code> y <code>/clientes/detalle/:id</code> .
MODIFICADO	<code>src/app/components/navbar/navbar.component.html</code>	Inclusión del botón de navegación "Clientes" en el encabezado principal del sitio.
MODIFICADO	<code>src/app/components/navbar/table-navbar/...</code>	Inclusión de los accesos a "Platos" y "Clientes" en el navbar de tablas administrativas.

4. Detalle de Implementación del Código

4.1. Servicio (`cliente.service.ts`)

El servicio expone una interfaz limpia basada en buenas prácticas de Angular:

- `getClientes(): Cliente[]` : Retorna todos los clientes (para vista de administración).
- `getClientesActivos(): Cliente[]` : Filtra solo aquellos con `activo !== false`.
- `getClienteById(id: number): Cliente | undefined` : Busca por clave primaria.
- `addCliente(cliente: Omit<Cliente, 'idCliente'>): void` : Autocalcula el siguiente ID incremental y agrega al listado.
- `updateCliente(id: number, datos): void` : Localiza el índice del cliente y actualiza sus propiedades.

- `desactivarCliente(id: number)` / `activarCliente(id: number)` : Alterna el estado lógico del registro.

Persistencia de apoyo (localStorage): Para evitar que al recargar la página durante una demostración se pierdan los clientes agregados o editados, el servicio guarda y recupera el estado en `localStorage`, manteniendo siempre como respaldo en memoria los datos originales.

4.2. Formulario Reactivo (`cliente-form.component.ts`)

Se empleó `ReactiveFormsModule` y `FormBuilder` aplicando validadores estrictos:

```
clienteForm = this.formBuilder.nonNullable.group({
  nombre: ['', [Validators.required, Validators.minLength(2), Validators.pattern(/^[a-zA-ZáéíóúÁÉÍÓÚñ
  apellido: ['', [Validators.required, Validators.minLength(2), Validators.pattern(/^[a-zA-ZáéíóúÁÉÍÓ
  correo: ['', [Validators.required, Validators.email]],
  telefono: ['', [Validators.required, Validators.minLength(7), Validators.pattern(/^[0-9+\- ]+$/)]],
  direccion: ['', [Validators.required, Validators.minLength(5)]],
  password: ['1234', [Validators.required, Validators.minLength(4)]]
});
```

Lógica de Edición vs Creación: En `ngOnInit`, se verifica si existe el parámetro `:id` en la ruta activa mediante `route.snapshot.paramMap.get('id')`. Si existe, se activa la bandera `isEdit = true` y se precargan los campos con `this.clienteForm.patchValue(...)`.

4.3. Tabla de Clientes (`clientes-tabla-page.component.html`)

Presenta los datos en una tabla estilizada con la paleta de colores del restaurante:

- Uso de la directiva moderna `@for (cliente of clientes; track cliente.idCliente)` con bloque `@empty` en caso de que no haya registros.
- Badges dinámicos para estado: verde para Activo y rojo suave para Inactivo.
- Botón inteligente que muestra "**Desactivar**" si el cliente está activo, o "**Activar**" si fue desactivado previamente.
- Botón superior destacado para redirigir a `/clientes/add`.

4.4. Detalle de Cliente (`cliente-detail.component.html`)

Muestra una tarjeta de perfil con avatar genérico, nombre completo, ID formateado, correo, teléfono, dirección y estado en el sistema, junto con botones para volver a la tabla o editar directamente.

5. Rutas Configuradas (`app.routes.ts`)

Ruta (Path)	Componente Asignado	Descripción
<code>/clientes</code>	<code>ClientesTablaPageComponent</code>	Listado tabular general con acciones de consulta, edición y baja lógica.
<code>/clientes/add</code>	<code>ClienteFormComponent</code>	Formulario vacío para registrar un nuevo cliente.
<code>/clientes/update/:id</code>	<code>ClienteFormComponent</code>	Formulario precargado para actualizar la información del cliente.
<code>/clientes/detalle/:id</code>	<code>ClienteDetailComponent</code>	Vista detallada individual del cliente seleccionado.

6. Validación y Compilación

Resultado de Compilación: El proyecto fue compilado utilizando el compilador oficial de Angular mediante el comando `ng build`. El resultado fue exitoso con **0 errores y 0 advertencias de TypeScript**, generando el bundle de producción sin conflictos.

7. Puntos Clave para el Equipo y Posibles Mejoras Futuras

1. **Transición a REST (Semana 10):** Cuando en la Semana 10 se creen los controladores REST en Spring Boot (ej. `@GetMapping("/api/clientes")`), lo único que deberá modificarse es el interior de `ClienteService` reemplazando el arreglo local por llamadas a `HttpClient.get()`, `HttpClient.post()`, etc. Los componentes no requerirán cambios mayores.
2. **Búsqueda y Filtros:** Como mejora opcional para la entrega final, se podría añadir una barra de búsqueda en tiempo real por nombre o correo en la tabla de clientes.

3. **Paginación:** Si la lista de clientes crece sustancialmente, se puede incorporar un paginador similar al de las comidas.